PRODUCT REQUIREMENTS DOCUMENT

Real-Time Chat Application

(Slack Clone)

Product Name
Real-Time Chat Application (Slack Clone)
Version
1.0.0
Product Type
Real-Time Collaborative Messaging Platform
Tech Stack
React · Node.js · Socket.io · Redis · MongoDB
Date
May 2026
Status
Draft

1. Product Overview

The Real-Time Chat Application is a full-featured, Slack-inspired messaging platform that enables teams to communicate instantly through channels and direct messages. Built on a modern WebSocket-driven architecture, it supports rich messaging features including file sharing, reactions, threads, and message editing — all delivered in real time.

The platform is designed for engineering teams and organisations that need an open, self-hosted communication layer with complete control over their data and infrastructure.

2. Target Users

- **Workspace Admins:** Create and manage workspaces, channels, and user roles. Full administrative access across the platform.
- **Channel Members:** Participate in channel conversations, share files, add reactions, and reply in threads.
- **Direct Message Users:** Engage in one-on-one or group private conversations outside of channels.

3. Core Features

3.1 Channels

- Create public and private channels with name and description
- Join, leave, and archive channels
- Channel membership management — invite and remove users
- Channel listing with unread message counts
- Admin-only channel deletion and settings management

3.2 Direct Messages

- One-on-one direct messaging between any two workspace users
- Group direct messages supporting multiple participants
- DM inbox with unread indicators and conversation previews
- Online / offline presence indicators on user avatars

3.3 File Sharing

- Upload images, documents, and other files within channels and DMs
- Inline image rendering and file preview support
- File metadata tracking — name, size, MIME type, uploader, timestamp
- Secure file access via signed URLs
- Multiple file attachments per message

3.4 Message Reactions

- React to any message with any emoji
- Multiple reactions per message from multiple users
- Reaction counts displayed per emoji with user tooltips
- Toggle reactions on/off with a single click
- Real-time reaction updates broadcast to all connected clients

3.5 Message Editing

- Edit your own messages after sending
- Edited messages display an 'edited' indicator with timestamp
- Real-time propagation of edits to all clients in the channel or DM
- Original message preserved in audit log

3.6 Thread Replies

- Reply to any message in a dedicated thread side-panel
- Thread reply count and participant avatars shown on parent message
- Thread notifications for participants who have engaged
- Threads supported in both channels and direct messages

4. Technical Specifications

4.1 Tech Stack

Layer	Technology	Purpose
Frontend	React	Component-based UI, real-time state management
Backend	Node.js + Express	REST API, business logic, middleware
Real-Time Transport	Socket.io	WebSocket events, rooms, broadcasting
Cache / Pub-Sub	Redis	Presence, typing indicators, pub-sub scaling
Database	MongoDB	Persistent storage for messages, users, channels

4.2 REST API Endpoints

Authentication Routes (/api/v1/auth/)

Method	Endpoint	Description	Auth
POST	/register	Register new user account	Public
POST	/login	Authenticate user, return JWT	Public
POST	/logout	Invalidate session	Secured
GET	/current-user	Get logged-in user info	Secured
POST	/refresh-token	Refresh access token	Public

Channel Routes (/api/v1/channels/)

Method	Endpoint	Description	Auth
GET	/	List all workspace channels	Secured
POST	/	Create a new channel	Admin
GET	/:channelId	Get channel details	Secured
PUT	/:channelId	Update channel info	Admin
DELETE	/:channelId	Delete channel	Admin

Method	Endpoint	Description	Auth
POST	/:channelId/join	Join a channel	Secured
POST	/:channelId/leave	Leave a channel	Secured
GET	/:channelId/members	List channel members	Secured
POST	/:channelId/members	Add member to channel	Admin
DELETE	/:channelId/members/:userId	Remove member	Admin

Message Routes (/api/v1/messages/)

Method	Endpoint	Description	Auth
GET	/:channelId	Fetch paginated channel messages	Secured
POST	/:channelId	Post a message (with optional files)	Secured
PUT	/:channelId/:messageId	Edit a message (owner only)	Secured
DELETE	/:channelId/:messageId	Delete a message	Secured
POST	/:channelId/:messageId/reactions	Add or toggle reaction	Secured
GET	/:channelId/:messageId/threads	Fetch thread replies	Secured
POST	/:channelId/:messageId/threads	Post a thread reply	Secured

Direct Message Routes (/api/v1/dm/)

Method	Endpoint	Description	Auth
GET	/	List all DM conversations	Secured
POST	/	Start a new DM or group DM	Secured
GET	/:dmId	Fetch messages in a DM conversation	Secured
POST	/:dmId	Send a message in a DM	Secured

File Routes (/api/v1/files/)

Method	Endpoint	Description	Auth
POST	/upload	Upload a file, return URL and metadata	Secured
GET	/:fileId	Retrieve file metadata	Secured
DELETE	/:fileId	Delete a file	Owner / Admin

4.3 Socket.io Events

Event Name	Direction	Description
message:send	Client → Server	Send a new message to a channel or DM
message:new	Server → Client	Broadcast new message to all room members
message:edit	Client → Server	Submit an edit to an existing message
message:updated	Server → Client	Broadcast updated message content
message:delete	Client → Server	Request deletion of a message
message:deleted	Server → Client	Notify all clients of deletion
reaction:toggle	Client → Server	Add or remove an emoji reaction
reaction:updated	Server → Client	Broadcast updated reaction state
thread:reply	Client → Server	Post a reply to a thread
thread:new	Server → Client	Broadcast new thread reply to participants
typing:start	Client → Server	Notify that user started typing
typing:stop	Client → Server	Notify that user stopped typing
typing:indicator	Server → Client	Broadcast typing indicator to room
presence:online	Server → Client	Notify workspace that user is online
presence:offline	Server → Client	Notify workspace that user went offline
channel:join	Client → Server	Join a Socket.io room for a channel
channel:leave	Client → Server	Leave a Socket.io room

5. Permission Matrix

Feature	Admin	Member
Create / Delete Channel	Yes	No
Send Messages	Yes	Yes
Edit Own Messages	Yes	Yes
Delete Any Message	Yes	No
Delete Own Message	Yes	Yes
Add Reactions	Yes	Yes

Feature	Admin	Member
Upload Files	Yes	Yes
Start Direct Message	Yes	Yes
Reply in Thread	Yes	Yes
Manage Channel Members	Yes	No
Archive Channel	Yes	No
View All Channels	Yes	Yes (public only)

6. Data Models

6.1 User

Field	Type	Description
_id	ObjectId	Unique identifier
username	String	Display name, unique per workspace
email	String	Login email, unique
passwordHash	String	Bcrypt-hashed password
avatarUrl	String	Profile picture URL
status	Enum	online | offline | away
createdAt	Date	Account creation timestamp

6.2 Channel

Field	Type	Description
_id	ObjectId	Unique identifier
name	String	Channel name (e.g. #general)
description	String	Optional channel description
type	Enum	public | private
members	ObjectId[]	References to User documents
createdBy	ObjectId	Reference to creator User
isArchived	Boolean	Soft-delete / archive flag
createdAt	Date	Creation timestamp

6.3 Message

Field	Type	Description
_id	ObjectId	Unique identifier
channelId	ObjectId	Parent channel or DM reference
senderId	ObjectId	Reference to sender User
content	String	Message text content
files	File[]	Array of attached file metadata objects

Field	Type	Description
reactions	Reaction[]	Array of emoji reaction objects
threadCount	Number	Count of thread replies
isEdited	Boolean	True if message has been edited
editedAt	Date	Timestamp of last edit
createdAt	Date	Original send timestamp

6.4 Reaction (Embedded)

Field	Type	Description
emoji	String	Emoji character or shortcode
userIds	ObjectId[]	Users who reacted with this emoji
count	Number	Derived count for quick rendering

6.5 File (Embedded / Collection)

Field	Type	Description
_id	ObjectId	Unique identifier
url	String	Secure access URL
mimeType	String	MIME type (e.g. image/png)
size	Number	File size in bytes
name	String	Original filename
uploadedBy	ObjectId	Reference to uploader User
createdAt	Date	Upload timestamp

7. Redis Architecture

- **Pub/Sub Channels:** Distribute Socket.io events across multiple Node.js instances, enabling horizontal scaling without sticky sessions.
- **Presence Tracking:** Store online/offline user status with TTL-based expiry to handle disconnects gracefully.
- **Typing Indicators:** Short-lived keys (2-second TTL) for per-user, per-channel typing state.
- **Token Caching:** Cache JWT refresh tokens for fast validation without database round-trips.

- **Unread Counts:** Cache per-user unread message counts per channel, updated on message receipt and cleared on channel visit.

8. Security Features

- JWT authentication with refresh token rotation and short-lived access tokens
- Role-based authorization middleware on all protected routes
- File upload validation — type whitelist and size limits enforced via Multer
- Rate limiting on message send and file upload endpoints to prevent abuse
- CORS configuration for cross-origin WebSocket and REST requests
- Input sanitisation on all message content to prevent XSS injection
- Signed, expiring URLs for file access — no public bucket exposure

9. Success Criteria

- Sub-100ms message delivery in real time via Socket.io under normal network conditions
- Scalable pub/sub architecture using Redis adapter supporting multi-instance deployment
- Full channel and direct message lifecycle — creation, messaging, archiving, deletion
- Rich messaging experience: reactions, threads, edits, and file attachments all functional
- Persistent, paginated message history with cursor-based pagination
- Presence and typing indicators updating in real time for all connected users
- Secure, role-based access enforced consistently across REST and Socket.io layers
- All user-uploaded files served via signed URLs with access expiry

10. Coding Agent Implementation Notes

The following additional specifications should be provided to a coding agent (Codex, Claude Code, Cursor, etc.) for end-to-end generation:

10.1 Folder Structure

```
server/
  src/
    controllers/    auth.js  channels.js  messages.js  dm.js  files.js
    models/         User.js  Channel.js  Message.js  File.js
    routes/         auth.js  channels.js  messages.js  dm.js  files.js
    sockets/        index.js  messageHandlers.js  presenceHandlers.js
    middleware/     auth.js  roles.js  upload.js  rateLimiter.js
    config/         db.js  redis.js  env.js
    utils/          jwt.js  errors.js
    app.js
  server.js
client/
  src/
    components/     Sidebar  ChannelView  MessageList  MessageInput
                    ThreadPanel  DMList  FileUpload  ReactionPicker
    hooks/          useSocket.js  usePresence.js  useMessages.js
    store/          authSlice.js  channelSlice.js  messageSlice.js
    pages/          Login  Register  Workspace
```

10.2 Environment Variables

Variable	Example Value	Purpose
PORT	5000	Express server port
MONGO_URI	mongodb://localhost:27017/chat	MongoDB connection string
REDIS_URL	redis://localhost:6379	Redis connection string
JWT_SECRET	your-secret-key	JWT signing secret
JWT_EXPIRES_IN	15m	Access token expiry
JWT_REFRESH_SECRET	your-refresh-secret	Refresh token secret
JWT_REFRESH_EXPIRES_IN	7d	Refresh token expiry
CLOUDINARY_URL	cloudinary://...	File storage (Cloudinary)
CLIENT_URL	http://localhost:3000	CORS allowed origin
MAX_FILE_SIZE_MB	10	Upload size limit in MB

10.3 Standardised Error Response Format

Field	Type	Example
success	Boolean	false
message	String	Channel not found
statusCode	Number	404
errors	Array (optional)	[{ field: 'name', msg: 'Required' }]

10.4 Pagination Strategy

- Use cursor-based pagination for message history
- Query parameter: ?before=<messageId>&limit=50
- Response includes nextCursor field (null when no more pages)
- Indexed on channelId + createdAt for query performance

10.5 File Storage

- Provider: Cloudinary (or AWS S3 as alternative)
- All uploads pass through Multer memory storage then streamed to provider
- Allowed MIME types: image/jpeg, image/png, image/gif, image/webp, application/pdf, text/plain
- Max file size: 10 MB per file, max 5 files per message